Deadlock Detection and Prevention in Business Process Models Using Petri Nets

Devansh Rai (IIB2024006), Geethika Gorakala (IIB2024024), Abhilasha Sharma (IIB2024501)

Business Process Management Project

Indian Institute of Information Technology Allahabad, Prayagraj, India

November 1, 2026

Abstract

Business process models drawn in BPMN can contain synchronisation errors, missing exception paths and conflicting resource use that leave a running case stuck before it reaches its end. Such deadlocks are hard to see by looking at a diagram. We present a six-step framework and a Python tool that (1) reads a BPMN 2.0 model, (2) maps it to a workflow Petri net, (3) detects deadlocks with a fast structural pattern check and an exhaustive reachability analysis, (4) explains the cause with a witness trace and siphon analysis, (5) applies a prevention rule, and (6) re-verifies the corrected model for soundness. We apply it to a Loan Approval process built up in six models of increasing complexity, plus five models from other domains. Of the 14 models, 7 are unsound: four deadlock through control-flow errors, two through circular waits on shared resources, and one completes improperly. Every one of them is corrected and re-verified as sound, and every analysis finishes in under 2 ms. Our soundness verdicts match the independent pm4py/WOFLAN checker on all 14 models. A timed simulation of the final model with branch probabilities completes 100% of 1000 cases with a mean cycle time of 62.4 minutes.

Index Terms— *BPMN, Petri nets, deadlock, soundness, reachability graph, siphon, workflow nets, business process verification*

1 Introduction

Organisations describe their workflows, such as loan approval, order handling and claim processing, as business process models, most often in the Business Process Model and Notation (BPMN) [9]. BPMN is easy to read but has no formal semantics, so a model can look correct and still contain behavioural errors [3]. The most harmful of these is a *deadlock*: a case reaches a state in which no activity can continue, although the process has not finished. In a loan process this means an application that is never approved or rejected.

Petri nets give a precise, graphical and mathematical model of states, activities and concurrency [6]. Once a BPMN model is mapped to a Petri net, deadlocks become reachable markings in which no transition is enabled, and they can be found mechanically [1].

Research question. *How can Petri nets be used to detect and prevent deadlocks in business process models while preserving correct workflow behaviour?*

Contributions. (i) A six-step framework that links BPMN modelling, Petri-net conversion, detection, diagnosis, correction and re-verification; (ii) a Python implementation with a Streamlit interface that combines the structural approach of [2], the reachability and siphon methods of [1] and the BPMN mapping with time and probability of [3]; (iii) a Loan Approval case study of six models, from a simple sequence to a timed model with shared resources, with before/after results.

2 Related Work

Hu, Zhu and Lei [1] review two ways to detect deadlocks in Petri nets: structural analysis of the net and analysis of the reachability tree. They note that structural results ignore the initial marking, while the reachability tree covers every reachable state, and they recommend combining the two. For prevention they summarise the basic-siphon method of Li and Zhou [7], which adds a control place for each basic strict minimal siphon.

Awad and Puhlmann [2] detect deadlocks directly on BPMN diagrams through graphical BPMN-Q queries that match error patterns, such as an XOR-split closed by an AND-join. Their method runs in polynomial time and points at the faulty sub-graph, but it is not complete.

Mutarraf et al. [3] transform BPMN XML into Petri nets automatically, extend the mapping with firing times on transitions and probabilities on gateways, and analyse the result in MATLAB/GPenSIM. Our mapping follows their rules and those of Dijkman et al. [5]. The correctness criterion we use is the soundness of workflow nets defined by van der Aalst [4].

3 Preliminaries

Petri net. A Petri net is $N = (P, T, F, W)$ with finite disjoint sets of places P and transitions T, arcs $F \subseteq (P \times T) \cup (T \times P)$ and weights $W : F \to \mathbb{N}^+$. A *marking* $M : P \to \mathbb{N}$ gives the number of tokens in each place. Table 1 lists the elements.

Firing rule. Transition t is enabled in M, written $M[t\rangle$, if $M(p) \geq W(p,t)$ for every input place $p \in {}^\bullet t$. Firing t gives $M'(p) = M(p) - W(p,t) + W(t,p)$.

Workflow net. A net with one source place i and one sink place o, initial marking $M_0 = [i]$ and final marking

Table 1: Petri net elements and their meaning in a process

Element	Meaning in a business process
Place (◯)	a condition or state, e.g. “credit checked”
Transition (□)	an activity or routing event
Token (●)	where the case currently is
Arc (→)	connects places and transitions

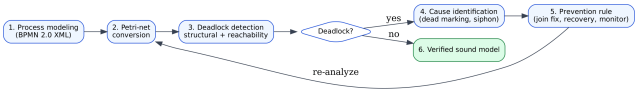


Figure 1: Petri-net-based deadlock detection and prevention framework.

$M_f = [o]$. Shared resources add places whose free units are present in both M_0 and M_f.

Deadlock. A reachable marking $M \in R(N, M_0)$ is *dead* if no transition is enabled. We call it a *deadlock* if, in addition, $M(o) = 0$, and an *improper completion* if $M(o) \geq 1$ but $M \neq M_f$:

$$M \in R(N, M_0), \quad M \neq M_f, \quad \nexists\, t \in T : M[t\rangle. \qquad (1)$$

Soundness [4]. A workflow net is sound if (1) from every reachable marking M_f can still be reached (option to complete), (2) whenever o is marked, no other token remains (proper completion), and (3) every transition can fire in some run (no dead transitions).

Siphon [1]. A non-empty set $S \subseteq P$ is a siphon if ${}^\bullet S \subseteq S^\bullet$, that is, every transition that puts a token into S also takes one from S. A siphon that becomes empty can never be marked again, so every transition that needs one of its places is dead from then on.

4 Proposed Methodology

Fig. 1 shows the six steps. A corrected model goes back through the whole pipeline, and it is accepted only when it is sound.

4.1 Step 1: Process modelling

Each model is drawn in BPMN 2.0 (Camunda Modeler or bpmn.io) and saved as XML. Extra information is written in the standard `documentation` field, so the files stay valid BPMN. We use the keys `resources: officer=1`, `acquire/release/uses: officer`, `time: 20` and `prob: 0.3`.

4.2 Step 2: BPMN to Petri net

Table 2 gives the mapping, following [3,5]. Every sequence flow becomes a place, so every token can be traced back to an arrow in the diagram.

4.3 Step 3: Deadlock detection

a) Structural check. Following [2], we search the BPMN graph for six patterns: P1, an XOR/OR-split whose branches are closed by an AND-join; P2, an AND-join at a loop entry; P3, an AND-split closed by an XOR-join (lack of synchronisation); P4, an element with no outgoing flow; P5, an unreachable element; and P6, two resources taken in opposite orders on different paths. For P1 we walk forward from every branch of a split. The pattern holds if one branch reaches an incoming flow a of the join but not another incoming flow b, while a second branch reaches b but not a.

b) Reachability analysis. Following [1], a breadth-first search builds the reachability graph from M_0, storing for each new marking the marking and transition it was reached from. Every dead marking other than M_f is reported together with its *witness trace*, the firing sequence that leads to it. A reverse search from M_f checks the option to complete, and the three soundness rules are evaluated. The search stops at 20,000 states, or when a place holds more than 50 tokens (treated as unbounded).

Table 2: BPMN to Petri net mapping

BPMN element	Petri net pattern
Sequence flow	one place
Start event	source place i (1 token) and a transition
End event	transition into sink place o
Task	one transition (input place → output place)
AND-split / AND-join	one transition with several output / input arcs
XOR-split / XOR-join	one transition per outgoing / incoming branch
OR-split	one transition per non-empty subset of branches
Shared resource	a place holding its free units; *acquire* adds an input arc, *release* an output arc
Element without outgoing flow	a “stuck” place that is never consumed

4.4 Step 4: Cause identification

For each dead marking we list the marked places and, for each one, the transition that is waiting and the input places it still lacks. Those transitions are mapped back to their BPMN elements and outlined in red in the diagram. For resource deadlocks we compute the minimal siphons, leaving out the source place, which empties by design. A siphon that is marked in M_0 but empty in a dead marking identifies the circular wait.

4.5 Step 5: Prevention rules

- *P1/P2:* make the join match the split by replacing the AND-join with an XOR-join.
- *P3:* replace the XOR-join with an AND-join.
- *P4 (missing recovery path):* redesign the model with a resubmission loop and a coordinated rejection path, so the join always receives a result.
- *P6 (circular wait):* for each problem siphon S, add a control place V_S [1, 7, 8]. With $d(t) =$

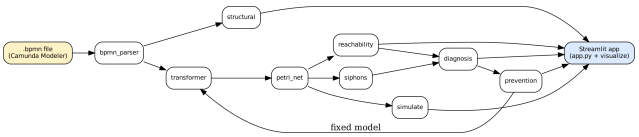


Figure 2: Module structure of the tool.

Table 3: Modules and their main functions

Module	What it does
`bpmn_parser`	`parse_bpmn()` reads the XML into nodes, flows, annotations
`transformer`	`bpmn_to_petri()` applies Table 2
`petri_net`	`enabled()`, `fire()` implement the firing rule
`structural`	`check()` finds patterns P1–P6
`reachability`	`analyse()` builds the graph, finds dead markings, checks soundness
`siphons`	`minimal_siphons()`, `problem_siphons()`
`diagnosis`	`explain()` says which transition waits for which place
`prevention`	`fix_gateways()`, `add_monitors()`
`simulate`	`run_cases()` runs the timed, probabilistic simulation
`app.py`	Streamlit interface: diagram, net, detection, fix, token game, simulation

$\sum_{p \in S} \big(W(t,p) - W(p,t)\big)$, we add an arc $t \to V_S$ of weight $d(t)$ when $d(t) > 0$ and an arc $V_S \to t$ of weight $-d(t)$ when $d(t) < 0$, and set $M_0(V_S) = M_0(S) - 1$. Then $M(V_S) - M(S) = -1$ holds in every reachable marking, so $M(S) \geq 1$ and S is never emptied.

The gateway rules are applied to the BPMN XML itself, so the corrected model can be opened again in the modeller.

4.6 Step 6: Re-analysis and verification

Steps 2–3 are repeated on the corrected model. It is accepted when it has no dead marking other than M_f, it is sound, and every path that completed correctly before still completes. For the final model we also run a timed simulation [3]. Transitions take their inputs when they start and give their outputs after their duration, XOR branches are drawn with the given probabilities, and we report the completion rate and cycle times.

5 Implementation

The tool is written in Python 3 with about 1,400 lines of code. Fig. 2 shows how the modules fit together. We implemented the core analysis ourselves so that every step can be explained. The library pm4py [10] is used only to cross-check our soundness results.

Input (dataset). The dataset is a set of 14 BPMN 2.0 XML process models: nine Loan Approval models (M1–M6 and three fixed versions) and five demonstration models from other domains. The tool also accepts a Petri net as JSON (places, transitions, arcs, initial and final marking).

Output. The Petri net and reachability graph with dead states in red; dead markings with witness traces; the cause report and siphons; the fixed BPMN file; a before/after table; and simulation statistics. The interface (Fig. 3) also offers a token game in which the user fires enabled transitions by hand.

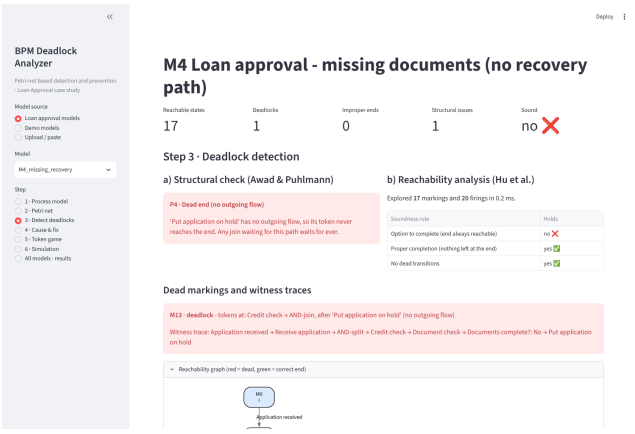


Figure 3: Detection step in the Streamlit application for model M4.

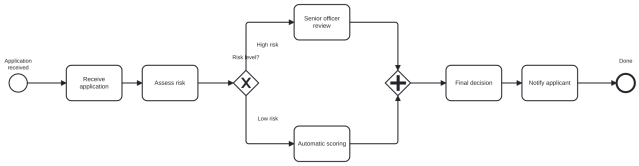


Figure 4: M3: the XOR-split "Risk level?" is closed by an AND-join.

6 Case Study and Results

6.1 Loan Approval models

- **M1** is a sequence: receive application, check eligibility, make decision, notify.
- **M2** runs the credit check and document check in parallel (AND-split/AND-join).
- **M3** routes by risk level with an XOR-split, but closes the branches with an AND-join (Fig. 4).
- **M4** lets the document check fail, but the "put on hold" path has no outgoing flow (the scenario of our proposal).
- **M5** runs a credit assessment and a fraud check in parallel. Both need the single loan officer and the single credit-system session, and they take them in opposite orders.
- **M6** combines all the corrections and adds durations, branch probabilities and a shared officer (Fig. 5).

6.2 Detection and correction results

Table 4 summarises all 14 models.

M3. Both dead markings hold a single token in front of the AND-join. For example, the trace *Receive application* → *Assess risk* → *Risk level?: High* → *Senior officer review* leaves the join waiting for the "Automatic scoring" branch, which was never started. Pattern P1 points at the same pair

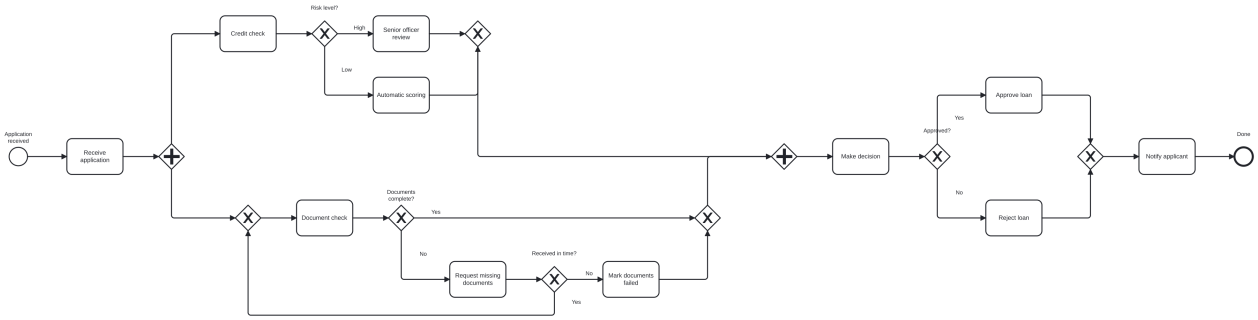


Figure 5: M6: final corrected Loan Approval model with resubmission loop, coordinated rejection and risk routing.

Table 4: Analysis results before and after correction (S = reachable states, D = deadlocks, I = improper completions)

Model	$\|P\|$	$\|T\|$	S	D	I	Sound	Pattern	Correction	Sound after	S after
M1 sequential	7	6	7	0	0	yes	–	–	–	–
M2 parallel	11	9	11	0	0	yes	–	–	–	–
M3 wrong join	12	11	8	2	0	no	P1	AND-join → XOR-join	yes	12
M4 missing recovery	14	12	17	1	0	no	P4	redesign (M4-fixed)	yes	32
M5 resource deadlock	17	13	20	1	0	no	P6	control place V_1 / fixed order	yes	19 / 19
M6 full timed	30	31	82	0	0	yes	–	–	–	–
D1 order fulfilment	13	12	13	0	0	yes	–	–	–	–
D2 insurance claim	10	9	7	2	0	no	P1	AND-join → XOR-join	yes	10
D3 leave request	10	9	25	0	1	no	P3	XOR-join → AND-join	yes	10
D4 hospital admission	16	12	19	1	0	no	P6	control place V_1	yes	18
D5 purchase order	10	9	3	1	0	no	P2	AND-join → XOR-join	yes	10

of gateways. Replacing the join with an XOR-join gives a sound net with 12 states.

M4. The dead marking has tokens in "Credit check → AND-join" and in the stuck place after "Put application on hold". The approval waits for a "documents complete" token that can never come (Fig. 6). No gateway change can fix this, so the model is redesigned as M4-fixed. Missing documents are requested and the check is repeated, and if they do not arrive the documents are marked as failed. Either way the AND-join receives a result, and a decision gateway then approves or rejects the loan. M4-fixed is sound with 32 states.

M5. The witness trace *AND-split → Assign loan officer → Open credit system (fraud)* reaches a dead marking in which each branch holds one resource and waits for the other. The analysis finds the siphon $S = \{r_{officer}, r_{credit}, p_5, p_8\}$, where p_5 and p_8 are the places in which a branch holds both resources. S holds 2 tokens in M_0 and none in the deadlock. The control place V_1, with 1 token, is taken by the first resource request of each branch and returned when the branch releases both resources (Fig. 7). So only one branch can be inside its critical section at a time, and the net becomes sound with 19 states. The alternative BPMN-level fix (M5-fixed: both branches take the officer first) is also sound. Pattern P6 detects the opposite ordering directly in the diagram.

Demo models. D2 and D5 show P1 and P2 in other domains. D3 is unsound without a deadlock: the XOR-join lets both parallel branches through, so "Update team calendar" runs twice and two tokens reach the end. This improper completion is caught by soundness rule 2. D4 repeats the circular-wait pattern with a bed and a doctor.

Cross-check. pm4py's WOFLAN check [10] reads the control flow only. On the control flow of all 14 models, its soundness verdict matches ours. All our analyses take under 2 ms per model.

6.3 Simulation of the final model

With the durations and probabilities shown in the model (e.g. 30% high-risk applications, 25% incomplete documents, 80% of requested documents received), 1000 simulated cases of M6 all complete. The mean cycle time is 62.4 minutes, the median 50 and the 90th percentile 78 (Fig. 8). In the uncorrected M5, when both branches start at the same moment, every simulated case deadlocks. With the control place, every case completes.

7 Discussion

The two detection methods complement each other, as [1] suggests. The structural check is instant and names the faulty gateways, but it is a heuristic: it can miss errors and, in unusual structures, raise false alarms. The reachability analysis is complete and gives a concrete witness trace, but the number of states can grow exponentially with concurrency (state explosion). Our largest model has 82 states, which is far below the limit. For much larger models, reduction rules or partial-order methods would be needed.

The corrections preserve correct behaviour. A gateway fix only removes runs that ended in a deadlock, and every run that completed before still completes. The siphon control place only delays one branch until the other releases its resources, so no activity is removed.

Limitations. OR-joins are approximated as XOR-joins. Event sub-processes, boundary events and message flows are not translated. The times and probabilities in M6 are assumed values, not data from a real bank. The recovery redesign for P4 cannot be automated, because it is a business

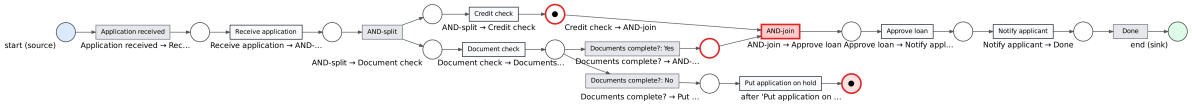


Figure 6: Petri net of M4 in its dead marking. Red: the waiting AND-join, the empty place it needs, and the stuck token.

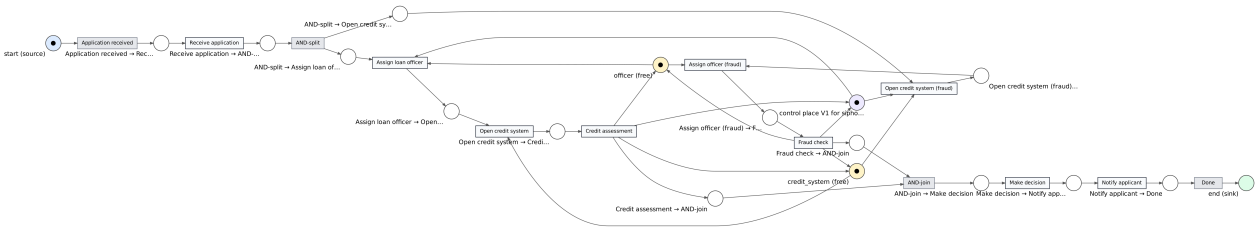


Figure 7: M5 with the siphon control place V_1 (purple) added by the prevention step.

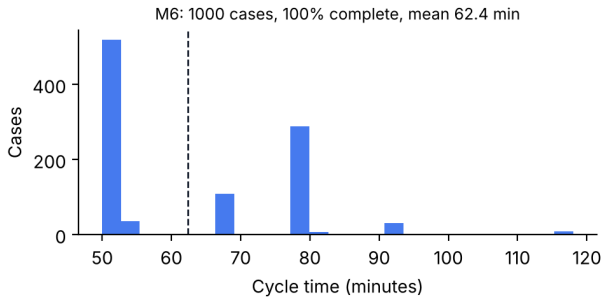


Figure 8: Cycle-time distribution of 1000 simulated M6 cases.

decision.

8 Conclusion and Future Work

We showed that Petri nets give a practical way to detect, explain and prevent deadlocks in BPMN models. Combining pattern-based structural checks with reachability and siphon analysis found every injected error in our 14 models, identified its cause, and confirmed each correction as sound. Future work includes full OR-join semantics, state-space reduction for large models, learning times and probabilities from event logs, and checking models while they are being drawn.

References

[1] W. Hu, Y. Zhu, and J. Lei, "The detection and prevention of deadlock in Petri nets," *Physics Procedia*, vol. 22, pp. 656–659, 2011, doi: 10.1016/j.phpro.2011.11.102.

[2] A. Awad and F. Puhlmann, "Structural detection of deadlocks in business process models," in *Business Information Systems (BIS 2008)*, LNBIP, vol. 7. Berlin, Germany: Springer, 2008, pp. 239–250, doi: 10.1007/978-3-540-79396-0_21.

[3] M. U. Mutarraf, K. Barkaoui, Z. Li, N. Wu, and T. Qu, "Transformation of business process model and notation models onto Petri nets and their analysis," *Advances in Mechanical Engineering*, vol. 10, no. 12, pp. 1–21, 2018, doi: 10.1177/1687814018808170.

[4] W. M. P. van der Aalst, "The application of Petri nets to workflow management," *J. Circuits, Systems and Computers*, vol. 8, no. 1, pp. 21–66, 1998.

[5] R. M. Dijkman, M. Dumas, and C. Ouyang, "Semantics and analysis of business process models in BPMN," *Information and Software Technology*, vol. 50, no. 12, pp. 1281–1294, 2008.

[6] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[7] Z. W. Li and M. C. Zhou, "Elementary siphons of Petri nets and their application to deadlock prevention in flexible manufacturing systems," *IEEE Trans. Syst., Man, Cybern. A*, vol. 34, no. 1, pp. 38–51, 2004.

[8] K. Yamalidou, J. Moody, M. Lemmon, and P. Antsaklis, "Feedback control of Petri nets based on place invariants," *Automatica*, vol. 32, no. 1, pp. 15–28, 1996.

[9] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0*, 2011.

[10] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst, "Process Mining for Python (PM4Py): Bridging the gap between process- and data science," in *Proc. ICPM Demo Track*, 2019, pp. 13–16.